Nama : M Luay Syam Daniyali

NIM : 2507036

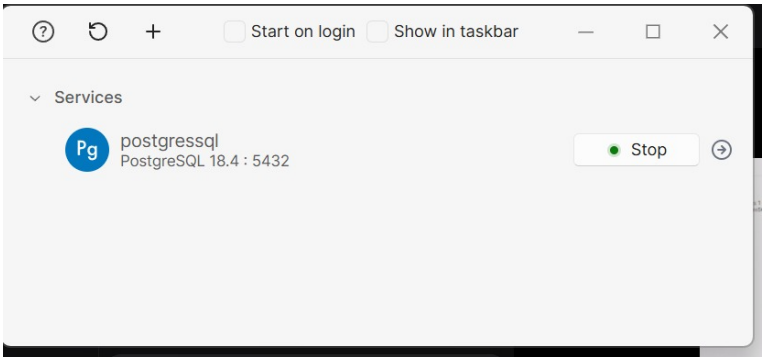
Kelas : Sistem Informasi B

Konfigurasi Index pada PostgreSQL

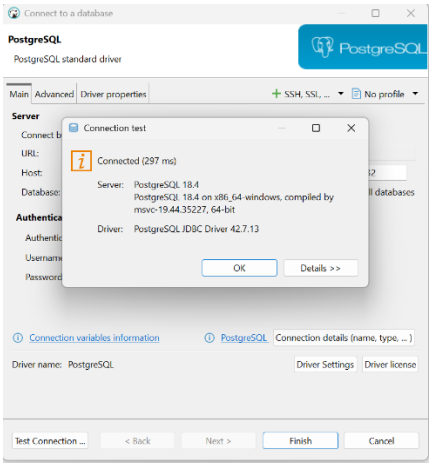
Dibuatnya ini untuk mengerjakan tugas Praktikum Rekayasa & Arsitektur Basis Data.

Langkah-langkah pengerjaan:

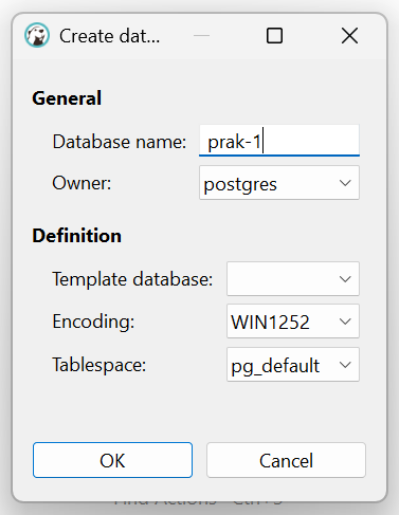
1. Buka DBngin, konek ke postgresql, lalu buat Database di DBngin, dan klik start, seperti Gambar di bawah



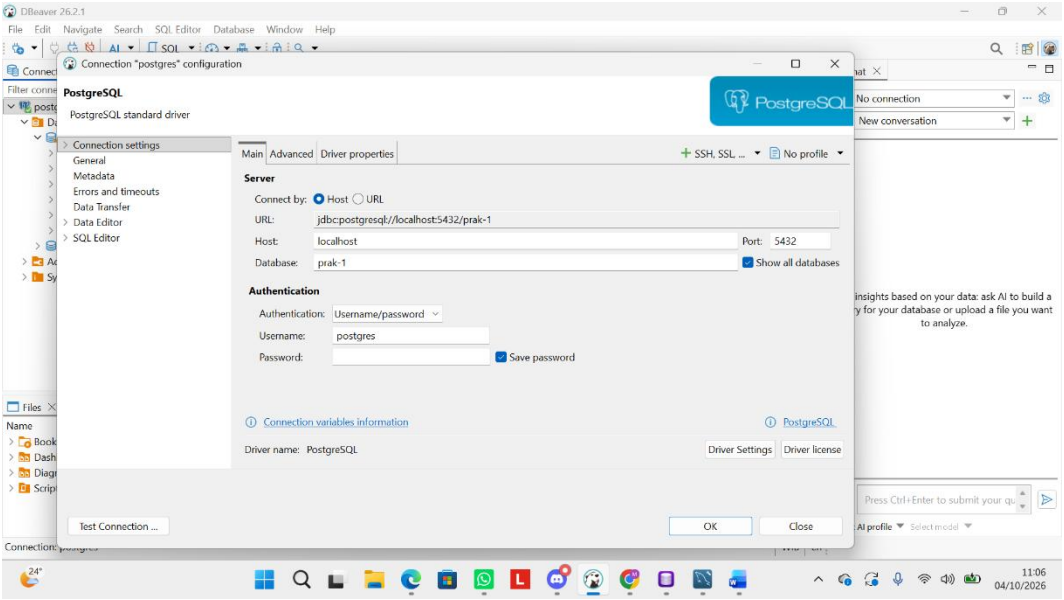
2. Buka DBeaver, lalu konek ke postgresql, lalu test Connection dulu, kalau aman lanjut finish.



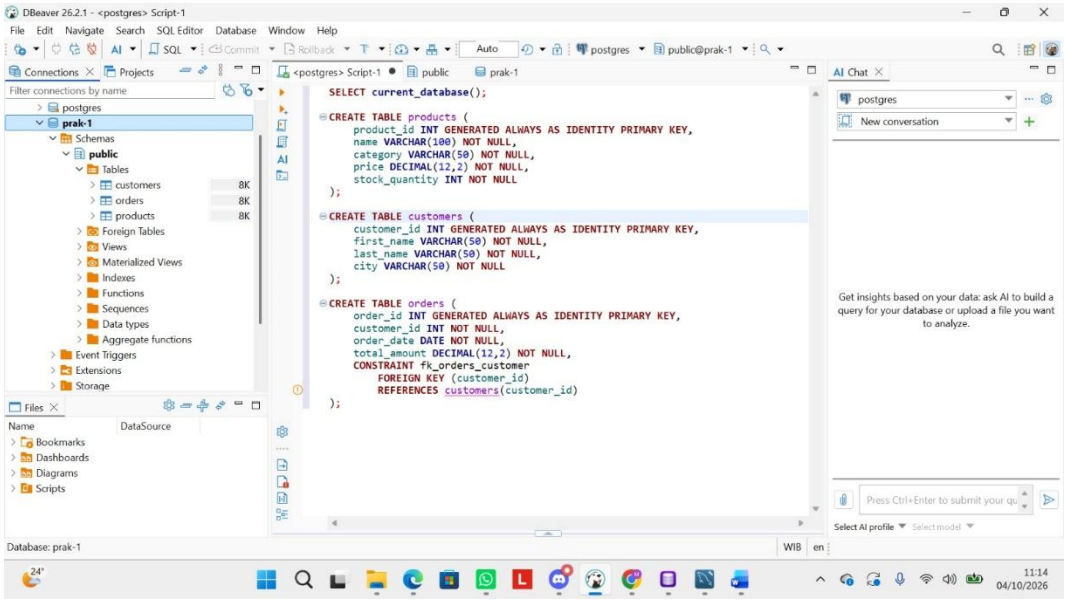
3. Lalu create database baru di dbeaver, agar sesuai nama Praktikum, contoh prak-1, lalu OK.



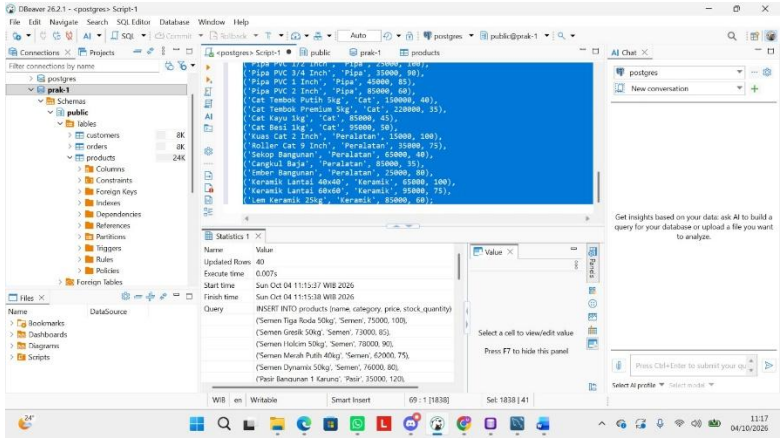
4. Lalu edit connection database default sebelumnya, Ganti nama database jadi prak-1, karena database prak-1 sudah eksis/ada, maka bisa konek loh ya, lalu OK.

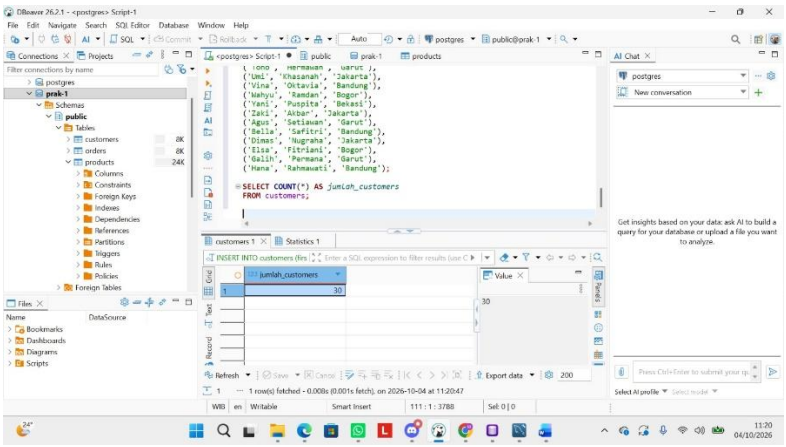


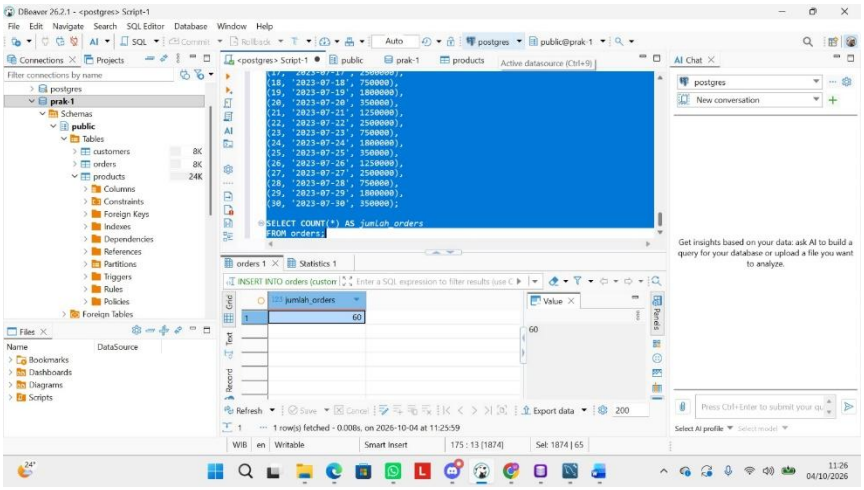
5. Lalu open SQL Editor, dan masukkan querry seperti digambar untuk tahap pembuatan Table.



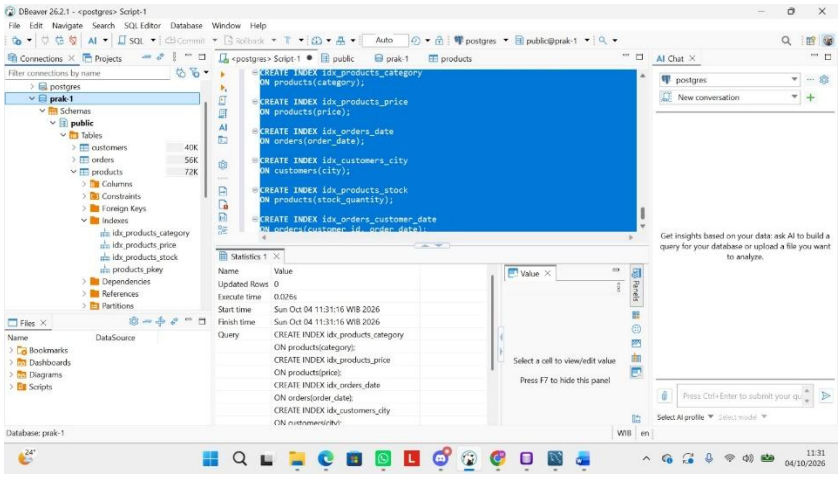
6. Lalu lanjut ke penginputan data set, 40 products, 30 costumers, dan 60 orders, isinya kami isi dengan data set dari toko matrial sebagai contoh.

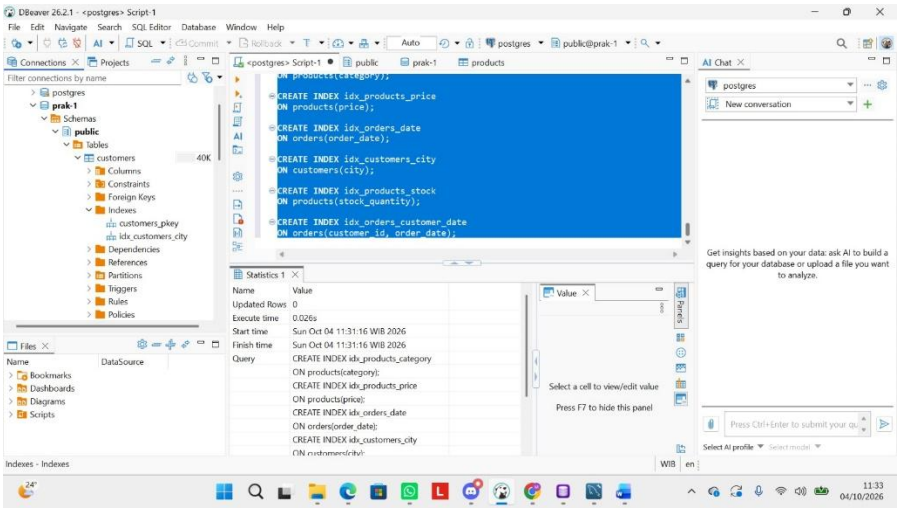


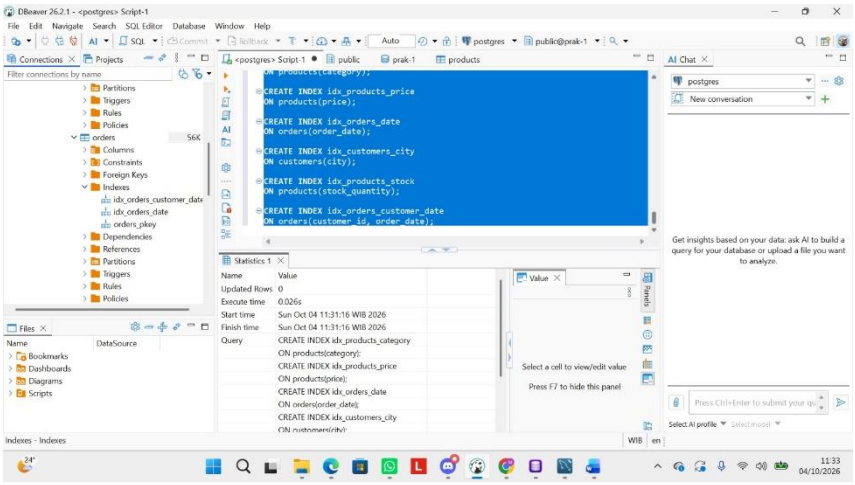




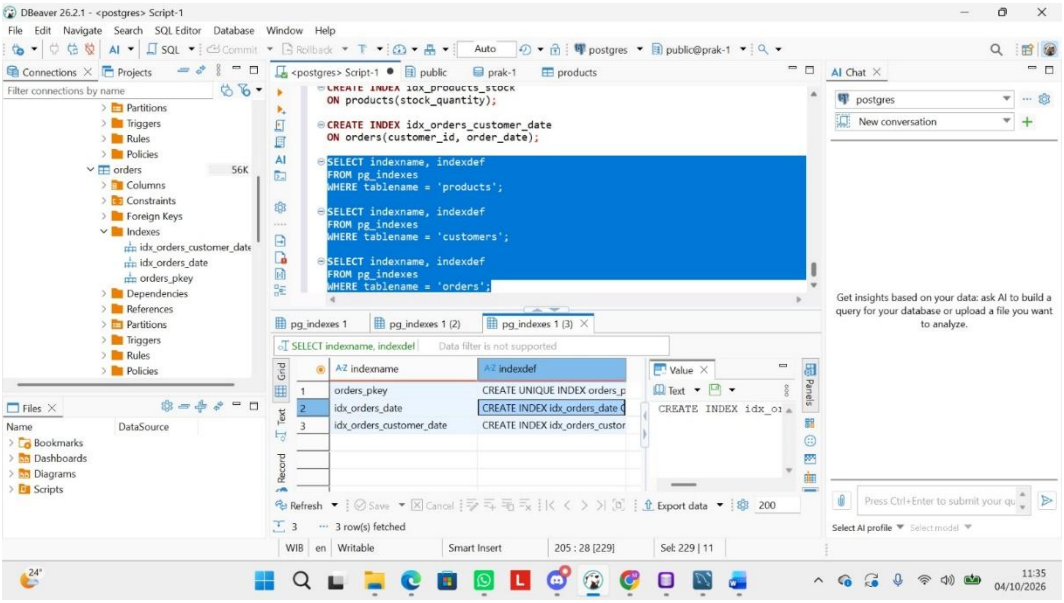
7. Sekarang kita lanjut ke tahap pembuatan index Products, Costumers dan Indexs, untuk Querry nya nanti ada di bawah.







8. Oke, setelah pembuatan indexs, lanjut ke pemeriksaan Indexs.



9. Oke sekarang saya akan menjawab pertanyaan Diskusi dari Modul

1. Apa fungsi index pada basis data?

Index berfungsi untuk membantu DBMS menemukan data tertentu secara lebih efisien tanpa harus membaca seluruh baris dalam tabel. Index dapat mempercepat proses pencarian data menggunakan kondisi WHERE, operasi JOIN, serta operasi tertentu seperti ORDER BY dan GROUP BY, bergantung pada struktur index yang digunakan.

2. Apakah setiap kolom harus diberikan index? Jelaskan.

Tidak, karena tidak semua kolom sering digunakan untuk pencarian atau pengolahan data. Index sebaiknya dibuat pada kolom yang sering digunakan dalam query, seperti kolom yang digunakan pada kondisi WHERE atau JOIN. Pembuatan index pada setiap kolom justru dapat menambah kebutuhan penyimpanan dan biaya pemeliharaan database.

3. Mengapa index dapat memperbesar biaya INSERT, UPDATE, dan DELETE?

Karena setiap kali data ditambahkan, diubah, atau dihapus, PostgreSQL juga perlu memperbarui struktur index yang berkaitan dengan data tersebut. Proses pemeliharaan ini membutuhkan sumber daya dan waktu tambahan, sehingga semakin banyak index yang dimiliki suatu tabel, semakin besar pula biaya yang diperlukan untuk operasi perubahan data.

4. Apa perbedaan single-column index dan composite index?

Single-column index adalah index yang dibuat pada satu kolom dalam tabel, sedangkan composite index adalah index yang dibuat menggunakan dua kolom atau lebih.

Contoh pada database toko material:

```
-- Single-column index
CREATE INDEX idx_products_category
ON products(category);

-- Composite index
CREATE INDEX idx_orders_customer_date
ON orders(customer_id, order_date);
```

Index pertama hanya menggunakan kolom category, sedangkan index kedua menggunakan kolom customer_id dan order_date secara bersamaan.

5. Mengapa urutan kolom pada composite index penting?

Urutan kolom pada composite index menentukan pola pencarian yang dapat didukung oleh index tersebut. Misalnya, index (customer_id, order_date) lebih sesuai untuk query yang mencari berdasarkan customer_id, atau berdasarkan customer_id sekaligus rentang order_date. Susunan (order_date, customer_id) tidak identik karena urutan kolom awalnya berbeda, sehingga pemanfaatannya untuk query juga dapat berbeda.

6. Bagaimana index dapat membantu operasi JOIN?

Index dapat membantu operasi JOIN dengan mempercepat pencarian baris yang memiliki nilai kolom penghubung yang sesuai pada tabel lain. Contohnya, pada tabel orders terdapat kolom customer_id sebagai foreign key yang menghubungkan data pesanan dengan tabel customers. Index pada kolom yang digunakan untuk menghubungkan kedua tabel dapat membantu PostgreSQL menemukan data yang cocok dengan lebih efisien. Namun, PostgreSQL tidak otomatis membuat index pada kolom foreign key.

7. Apa risiko membuat terlalu banyak index pada suatu tabel?

Terlalu banyak index dapat meningkatkan penggunaan ruang penyimpanan dan biaya pemeliharaan database. Setiap operasi INSERT, UPDATE, dan DELETE juga dapat membutuhkan proses tambahan untuk memperbarui index yang terkait. Oleh karena itu, index perlu dibuat sesuai kebutuhan query dan tidak berlebihan agar pengelolaan database tetap efisien.

KESIMPULAN

Berdasarkan praktikum konfigurasi indexing pada PostgreSQL yang telah dilakukan, database toko material berhasil dirancang dengan tiga tabel, yaitu products, customers, dan orders, yang saling berhubungan untuk menyimpan data produk, pelanggan, dan transaksi. Dataset yang digunakan terdiri dari 40 data produk material bangunan, 30 data pelanggan, dan 60 data pesanan sesuai dengan ketentuan praktikum.

Pada praktikum ini, telah dirancang *single-column index* pada kolom category dan price di tabel products, serta *composite index* pada kolom customer_id dan order_date di tabel orders. Index tersebut disesuaikan dengan pola query yang digunakan, seperti pencarian produk berdasarkan kategori atau rentang harga, serta pencarian pesanan berdasarkan pelanggan dan tanggal transaksi. Pemeriksaan melalui pg_indexes digunakan untuk melihat index yang telah dibuat pada tabel.

Dari hasil praktikum ini, dapat disimpulkan bahwa index merupakan salah satu komponen penting dalam pengelolaan basis data karena dapat membantu proses pencarian dan pengambilan data menjadi lebih efisien. Namun, pembuatan index perlu disesuaikan dengan kebutuhan query karena setiap index juga membutuhkan ruang penyimpanan dan pemeliharaan ketika data ditambahkan, diubah, atau dihapus. Dengan demikian, pemilihan jenis dan kolom index yang tepat dapat membantu pengelolaan database toko material secara lebih terstruktur dan efisien.

REFLEKSI

1. Index mana saja yang berhasil dibuat, dan pada kolom apa?

Index yang dibuat pada database toko material terdiri dari dua *single-column index* dan satu *composite index*. *Single-column index* yang dibuat adalah idx_products_category pada kolom category dan idx_products_price pada kolom price di tabel products. Sementara itu, *composite index* yang dibuat adalah idx_orders_customer_date pada kolom customer_id dan order_date di tabel orders. Index tersebut dibuat untuk mendukung pencarian data berdasarkan kategori produk, harga, serta pelanggan dan tanggal pesanan.

2. Apa konsekuensi jika sebuah tabel memiliki terlalu banyak index?

Jika sebuah tabel memiliki terlalu banyak index, kebutuhan ruang penyimpanan database akan meningkat karena setiap index membutuhkan ruang tersendiri. Selain itu, operasi INSERT, UPDATE, dan DELETE dapat menjadi lebih lambat karena PostgreSQL harus memperbarui index yang berkaitan dengan perubahan data. Oleh karena itu, jumlah index perlu disesuaikan dengan kebutuhan agar tidak menimbulkan biaya pemeliharaan yang berlebihan.

3. Bagaimana menentukan apakah sebuah kolom layak diberikan index?

Kolom layak diberikan index apabila sering digunakan dalam query, terutama pada kondisi WHERE, operasi JOIN, atau pengurutan data menggunakan ORDER BY. Pemilihan index juga perlu mempertimbangkan pola pencarian dan kebutuhan aplikasi. Pada database toko material, kolom category dan price diberikan index karena digunakan untuk pencarian produk berdasarkan kategori dan rentang harga. Dengan demikian, index sebaiknya dibuat berdasarkan kebutuhan query dan bukan pada semua kolom tabel.

Lalu Untuk Full Code/Querry nya ini:

Termasuk dari:

• pembuatan database

• pembuatan tabel

• pengisian dataset

• pembuatan index

• pemeriksaan index.

```
SELECT current_database();

CREATE TABLE products (
    product_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    category VARCHAR(50) NOT NULL,
    price DECIMAL(12,2) NOT NULL,
    stock_quantity INT NOT NULL
);

CREATE TABLE customers (
    customer_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    city VARCHAR(50) NOT NULL
);

CREATE TABLE orders (
    order_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    customer_id INT NOT NULL,
    order_date DATE NOT NULL,
    total_amount DECIMAL(12,2) NOT NULL,
    CONSTRAINT fk_orders_customer
        FOREIGN KEY (customer_id)
        REFERENCES customers(customer_id)
);

INSERT INTO products (name, category, price, stock_quantity) VALUES
('Semen Tiga Roda 50kg', 'Semen', 75000, 100),
('Semen Gresik 50kg', 'Semen', 73000, 85),
('Semen Holcim 50kg', 'Semen', 78000, 90),
('Semen Merah Putih 40kg', 'Semen', 62000, 75),
('Semen Dynamix 50kg', 'Semen', 76000, 80),
('Pasir Bangunan 1 Karung', 'Pasir', 35000, 120),
('Pasir Beton 1 Karung', 'Pasir', 45000, 90),
('Pasir Pasang 1 Karung', 'Pasir', 30000, 110),
('Batu Split 1 Karung', 'Batu', 40000, 100),
('Batu Kerikil 1 Karung', 'Batu', 30000, 95),
('Bata Merah', 'Bata', 1200, 2000),
('Bata Ringan Hebel 10cm', 'Bata', 8500, 500),
('Batako Press', 'Bata', 4500, 800),
('Paving Block Abu-Abu', 'Bata', 2500, 1000),
('Besi Beton 8mm', 'Besi', 55000, 100),
('Besi Beton 10mm', 'Besi', 75000, 90),
('Besi Beton 12mm', 'Besi', 105000, 80),
('Besi Hollow 4x4', 'Besi', 85000, 70),
('Kawat Bendrat 1kg', 'Besi', 22000, 120),
('Paku 5cm 1kg', 'Besi', 25000, 100),
('Seng Gelombang', 'Atap', 65000, 80),
('Genteng Tanah Liat', 'Atap', 3500, 1000),
('Genteng Metal', 'Atap', 45000, 150),
('Atap Spandek 3 Meter', 'Atap', 150000, 50),
('Pipa PVC 1/2 Inch', 'Pipa', 25000, 100),
('Pipa PVC 3/4 Inch', 'Pipa', 35000, 90),
('Pipa PVC 1 Inch', 'Pipa', 45000, 85),
('Pipa PVC 2 Inch', 'Pipa', 85000, 60),
('Cat Tembok Putih 5kg', 'Cat', 150000, 40),
```

```
('Cat Tembok Premium 5kg', 'Cat', 220000, 35),
('Cat Kayu 1kg', 'Cat', 85000, 45),
('Cat Besi 1kg', 'Cat', 95000, 50),
('Kuas Cat 2 Inch', 'Peralatan', 15000, 100),
('Roller Cat 9 Inch', 'Peralatan', 35000, 75),
('Sekop Bangunan', 'Peralatan', 65000, 40),
('Cangkul Baja', 'Peralatan', 85000, 35),
('Ember Bangunan', 'Peralatan', 25000, 80),
('Keramik Lantai 40x40', 'Keramik', 65000, 100),
('Keramik Lantai 60x60', 'Keramik', 95000, 75),
('Lem Keramik 25kg', 'Keramik', 85000, 60);

SELECT COUNT(*) AS jumlah_products
FROM products;

SELECT * FROM products;

INSERT INTO customers (first_name, last_name, city)
VALUES
('Andi', 'Pratama', 'Jakarta'),
('Budi', 'Santoso', 'Bandung'),
('Citra', 'Lestari', 'Jakarta'),
('Deni', 'Kurniawan', 'Bogor'),
('Eka', 'Saputra', 'Bandung'),
('Fajar', 'Hidayat', 'Garut'),
('Gita', 'Permata', 'Jakarta'),
('Hendra', 'Wijaya', 'Bekasi'),
('Indah', 'Sari', 'Bandung'),
('Joko', 'Setiawan', 'Garut'),
('Kiki', 'Ramadhan', 'Bogor'),
('Lina', 'Marlina', 'Jakarta'),
('Maman', 'Suryana', 'Garut'),
('Nina', 'Anggraini', 'Bandung'),
('Oki', 'Firmansyah', 'Bekasi'),
('Putri', 'Amelia', 'Jakarta'),
('Rian', 'Maulana', 'Bogor'),
('Sinta', 'Nuraini', 'Bandung'),
('Tono', 'Hermawan', 'Garut'),
('Umi', 'Khasanah', 'Jakarta'),
('Vina', 'Oktavia', 'Bandung'),
('Wahyu', 'Ramdan', 'Bogor'),
('Yani', 'Puspita', 'Bekasi'),
('Zaki', 'Akbar', 'Jakarta'),
('Agus', 'Setiawan', 'Garut'),
('Bella', 'Safitri', 'Bandung'),
('Dimas', 'Nugraha', 'Jakarta'),
('Elsa', 'Fitriani', 'Bogor'),
('Galih', 'Permana', 'Garut'),
('Hana', 'Rahmawati', 'Bandung');

SELECT COUNT(*) AS jumlah_customers
FROM customers;

INSERT INTO orders (customer_id, order_date, total_amount)
VALUES
(1, '2023-06-01', 2500000),
(2, '2023-06-02', 750000),
(3, '2023-06-03', 1250000),
(4, '2023-06-04', 350000),
(5, '2023-06-05', 1800000),
(6, '2023-06-06', 2500000),
(7, '2023-06-07', 750000),
(8, '2023-06-08', 1250000),
(9, '2023-06-09', 350000),
(10, '2023-06-10', 1800000),
(11, '2023-06-11', 2500000),
(12, '2023-06-12', 750000),
(13, '2023-06-13', 1250000),
(14, '2023-06-14', 350000),
(15, '2023-06-15', 1800000),
(16, '2023-06-16', 2500000),
(17, '2023-06-17', 750000),
```

```
(18, '2023-06-18', 1250000),
(19, '2023-06-19', 350000),
(20, '2023-06-20', 1800000),
(21, '2023-06-21', 2500000),
(22, '2023-06-22', 750000),
(23, '2023-06-23', 1250000),
(24, '2023-06-24', 350000),
(25, '2023-06-25', 1800000),
(26, '2023-06-26', 2500000),
(27, '2023-06-27', 750000),
(28, '2023-06-28', 1250000),
(29, '2023-06-29', 350000),
(30, '2023-06-30', 1800000),
(1, '2023-07-01', 1250000),
(2, '2023-07-02', 2500000),
(3, '2023-07-03', 750000),
(4, '2023-07-04', 1800000),
(5, '2023-07-05', 350000),
(6, '2023-07-06', 1250000),
(7, '2023-07-07', 2500000),
(8, '2023-07-08', 750000),
(9, '2023-07-09', 1800000),
(10, '2023-07-10', 350000),
(11, '2023-07-11', 1250000),
(12, '2023-07-12', 2500000),
(13, '2023-07-13', 750000),
(14, '2023-07-14', 1800000),
(15, '2023-07-15', 350000),
(16, '2023-07-16', 1250000),
(17, '2023-07-17', 2500000),
(18, '2023-07-18', 750000),
(19, '2023-07-19', 1800000),
(20, '2023-07-20', 350000),
(21, '2023-07-21', 1250000),
(22, '2023-07-22', 2500000),
(23, '2023-07-23', 750000),
(24, '2023-07-24', 1800000),
(25, '2023-07-25', 350000),
(26, '2023-07-26', 1250000),
(27, '2023-07-27', 2500000),
(28, '2023-07-28', 750000),
(29, '2023-07-29', 1800000),
(30, '2023-07-30', 350000);

SELECT COUNT(*) AS jumlah_orders
FROM orders;

CREATE INDEX idx_products_category
ON products(category);

CREATE INDEX idx_products_price
ON products(price);

CREATE INDEX idx_orders_date
ON orders(order_date);

CREATE INDEX idx_customers_city
ON customers(city);

CREATE INDEX idx_products_stock
ON products(stock_quantity);

CREATE INDEX idx_orders_customer_date
ON orders(customer_id, order_date);

SELECT indexname, indexdef
FROM pg_indexes
WHERE tablename = 'products';

SELECT indexname, indexdef
FROM pg_indexes
WHERE tablename = 'customers';
```

```
SELECT indexname, indexdef
FROM pg  indexes
WHERE tablename = 'orders';
```